

Diário de Hábitos — Guia do Código (Check 2)

Sep 29, 2026 · @Pedro e Malu

Este guia explica, em palavras simples, como o projeto `diario_de_habitos1` está organizado e como cada parte funciona — com foco especial em como excluir um hábito, que é a tarefa nova do Check 2.

Por que dividir em pastas

A ideia central: **cada camada só pode conhecer a de baixo, nunca o contrário.**

```
Interface (ui/) → conhece → Domínio (dominio/) → conhece → Dados (dados/)
```

Pensa assim: se um dia você trocar a tela de lista por um calendário, só a pasta `ui/` muda. Se trocar de onde vêm os dados (de memória para um banco de verdade), só a pasta `dados/` muda. O `dominio/` — a regra do produto, o "o que é um hábito e como ele se comporta" — não muda em nenhum dos dois casos, porque ele não depende de tela nem de banco.

O teste que o professor ensinou: **"isto sobrevive se a tela mudar?"** Se sim, não é interface — é domínio ou dado.

As três pastas, e o que cada uma guarda

<code>lib/</code>	
<code> — main.dart</code>	→ cria tudo e liga as peças
<code> — ui/</code>	→ telas e widgets (o que o usuário vê e toca)
<code> — tela_habitos.dart</code>	→ a lista
<code> — tela_novo_habito.dart</code>	→ o formulário
<code> — tela_detalhe.dart</code>	→ o detalhe + botão Excluir
<code> — dominio/</code>	→ o modelo e as regras do produto
<code> — habito.dart</code>	→ o que é um hábito
<code> — habitos_store.dart</code>	→ guarda a lista e avisa quem depende dela
<code> — dados/</code>	→ de onde vem e para onde vai o dado
<code> — habitos_repositorio.dart</code>	→ hoje guarda em memória (RAM)

ui/ (interface): telas e widgets. É o que muda se você redesenhar o app ou trocar de plataforma. Não guarda regra nenhuma — só mostra o que a loja (`dominio/`) entrega e chama os métodos dela quando o usuário toca em algo.

dominio/ (domínio): o modelo (`Habito`) e a loja (`HabitosStore`). É o "produto" em si — sobrevive a qualquer troca de tela ou de banco de dados.

dados/ (dados): o repositório. É só isso: de onde o dado vem e para onde ele vai. Hoje é uma lista em memória; poderia ser um banco SQLite ou uma API, e nenhuma tela precisaria saber a diferença.

O modelo `Habito` (`dominio/habito.dart`)

```
class Habito {
  final String nome;
  final String meta;
  final IconData icone;
  final String imageUrl;

  const Habito(this.nome, this.meta, this.icone, this.imageUrl);
}
```

É uma classe simples, só de dados — não tem nenhuma lógica dentro. Todos os campos são `final`: depois que um `Habito` é criado, ele nunca muda. Se você quer um hábito diferente, cria outro objeto, não edita o existente.

Fica em `dominio/` porque esse objeto "sobrevive" mesmo que a tela mude completamente — é o produto, não a forma de mostrar ele.

O repositório (`dados/habitos_repositorio.dart`)

```
class HabitosRepositorio {
  final List<Habito> _memoria = [ ...hábitos iniciais... ];

  Future<List<Habito>> carregar() async => List.of(_memoria);
  Future<void> salvar(Habito h) async { _memoria.add(h); }
  Future<void> remover(Habito h) async { _memoria.remove(h); }
}
```

Pensa nele como o **caderno onde os dados realmente ficam guardados**. `_memoria` é privada (o `_` na frente trava o acesso de fora) — só este arquivo mexe nela diretamente.

Os três métodos são `Future` e `async`, mesmo sendo instantâneos hoje. Isso simula um acesso real a dados (banco, servidor): se um dia trocar por SQLite ou uma API, só o **corpo** desses métodos muda — a assinatura continua igual, e ninguém que usa o repositório precisa saber se a resposta veio na hora ou depois de dois segundos.

`carregar()` devolve `List.of(_memoria)`, uma **cópia**, não a lista original — evita que alguém mexa direto na `_memoria` por fora.

A `HabitosStore` (`dominio/habitos_store.dart`) — a loja

```
class HabitosStore extends ChangeNotifier {
  HabitosStore(this._repo);
  final HabitosRepositorio _repo;
  List<Habito> _habitos = [];

  List<Habito> get habitos => List.unmodifiable(_habitos);

  Future<void> carregar() async {
    _habitos = await _repo.carregar();
    notifyListeners();
  }

  Future<void> adicionar(Habito h) async {
    await _repo.salvar(h);
    _habitos = await _repo.carregar();
    notifyListeners();
  }

  Future<void> remover(Habito h) async {
    await _repo.remover(h);
    _habitos = await _repo.carregar();
    notifyListeners();
  }
}
```

Pensa nela como **um secretário**: ninguém mexe no caderno (o repositório) diretamente — todo mundo pede pro secretário, e só ele tem permissão de escrever ou apagar algo ali.

- **`extends ChangeNotifier`**: dá à classe o poder de avisar quem está "escutando" ela.

- **HabitosStore(this._repo)** : o repositório é entregue de fora (no `main.dart`), não criado aqui dentro.
- **get habitos => List.unmodifiable(...)** : devolve uma lista travada — ninguém consegue alterar por fora, só os métodos desta classe.
- **Padrão dos três métodos**: pede a ação pro repositório, espera (`await`), **recarrega tudo de novo** do repositório, e só então chama `notifyListeners()` — a fonte da verdade é sempre o repositório, nunca a lista local por si só.
- **notifyListeners()** : é o "avisa todo mundo que estava esperando notícia". Quem estiver com `context.watch<HabitosStore>()` numa tela é automaticamente reconstruído quando isso é chamado.

As três telas (ui/)

tela_habitos.dart (a lista) — é `StatefulWidget` porque precisa rodar `context.read<HabitosStore>().carregar()` uma vez, no `initState`, pra pedir os dados assim que a tela abre. No `build`, usa `context.watch<HabitosStore>().habitos` — toda vez que a loja avisa (`notifyListeners`), essa tela se redesenha sozinha, sem código extra. O `onTap` de cada item navega para `TelaDetalhe(habito: habitos[i])`, passando o objeto inteiro.

tela_novo_habito.dart (o formulário) — `StatefulWidget` com `TextEditingController` para cada campo. Ao salvar, chama `context.read<HabitosStore>().adicionar(novo)` (com `await`, porque agora é assíncrono), depois `if (!context.mounted) return;` (checagem de segurança pós-`await`) e por fim `Navigator.pop(context)` sem devolver nada — quem escreveu o dado foi a loja, não a navegação.

tela_detalhe.dart (o detalhe + Excluir) — `StatelessWidget`, recebe `required this.habito`. Tem um `IconButton` de lixeira dentro de `actions:` na `AppBar`. É o assunto da próxima seção, em detalhe.

O que veio da Prática 5 (Interfaces II)

Essa aula deu a base de tudo que veio depois:

- **ListView.builder** : constrói só os itens visíveis na tela (não todos de uma vez), usando `itemCount` e `itemBuilder`. É o que está em `tela_habitos.dart` até hoje.

- **setState** : o "aviso" que diz ao Flutter "o estado mudou, redesenha". Sem ele, mudar uma variável não muda nada na tela — o **build** só roda de novo quando alguém pede.
- **TextEditingController** + **Form / validator** : capturar o que o usuário digita e recusar o que não serve antes de aceitar. O **validator** devolve uma mensagem de erro (se inválido) ou **null** (se válido) — não um **true / false** direto.

No projeto atual, **setState** **não aparece mais** na tela da lista, porque essa responsabilidade passou para a **HabitosStore** (Prática 7). Mas o formulário (`tela_novo_habito.dart`) ainda usa exatamente o padrão de **TextEditingController** + **Form** + **validator** ensinado aqui.

O que veio da Prática 6 (Navegação)

- **A navegação é uma pilha**: **push** empilha uma tela nova; **pop** desempilha (volta). A tela de baixo continua existindo, com o estado intacto.
- **Passar dado para a tela seguinte** é passar parâmetro para o construtor, como em qualquer classe Dart: `TelaDetalhe({required this.habito})`.
- **Trazer resultado de volta com `await Navigator.push<T>(...)`** era o padrão *antes* de existir a **HabitosStore**. Hoje esse padrão **não é mais usado**: como a store guarda os dados, a tela não precisa mais esperar um retorno pela navegação — ela só espera o `notifyListeners()` da store.

No seu código atual, a navegação ficou mais simples do que na Prática 6 original, exatamente porque a Prática 7 tirou a necessidade de devolver dado pela pilha de navegação.

Excluir um hábito — o fluxo completo, passo a passo

Esta é a tarefa do Check 2. Vale entender cada etapa, porque é a mais provável de ser perguntada.

A analogia: pensa na **HabitosRepositorio** como um **caderno** onde os dados ficam escritos, e a **HabitosStore** como o **secretário** — o único que tem permissão de mexer no caderno. Nenhuma tela mexe direto no caderno; toda tela pede pro secretário.

Passo a passo do que acontece quando você aperta a lixeira na **TelaDetalhe :**

1. O **IconButton** na **AppBar** dispara seu **onPressed** (que é **async**).

2. `context.read<HabitosStore>().remove(habito)` — a tela **pede** pro secretário (a store) remover, passando o `Habito` inteiro (não um índice, porque a tela de detalhe só tem o objeto, não sabe a posição dele na lista).
3. Dentro de `HabitosStore.remove(h)`: primeiro `await _repo.remove(h)` — o secretário vai até o caderno e apaga aquele hábito de lá.
4. Depois, `_habitos = await _repo.carregar()` — o secretário **relê o caderno inteiro**, pra garantir que a lista local reflita exatamente o que ficou gravado.
5. `notifyListeners()` — o secretário **avisa todo mundo que está esperando notícia dele**.
6. De volta na `TelaDetalhe`: `if (!context.mounted) return;` — checa se a tela ainda existe (pode ter sido fechada enquanto esperava os `await s`).
7. `Navigator.pop(context)` — só agora a tela fecha e volta pra lista.
8. A `TelaHabitos`, que já estava com `context.watch<HabitosStore>()` no `build`, **já tinha se redesenhado sozinha** assim que o passo 5 aconteceu — antes mesmo do `pop` rodar. Por isso a lista aparece sem o hábito, sem nenhum código extra pra "atualizar a tela".

A regra de ouro do slide do Check: *"a lista é alterada pela classe que a guarda, nunca pelo widget do botão"*. O `IconButton` nunca toca na lista — ele só manda a store agir.

Por que `remove` recebe o `Habito` inteiro, e não um índice como antes? Porque a `TelaDetalhe` só tem o objeto (recebido pelo construtor), não sabe em que posição da lista ele está. `_habitos.remove(h)` (no repositório) localiza e remove por igualdade de objeto.

O que aconteceria sem o `notifyListeners()`? O hábito seria removido do caderno de verdade, mas nenhuma tela saberia — a lista continuaria mostrando o hábito antigo até fechar e reabrir o app.

Onde a exclusão "realmente acontece"? Se o professor perguntar isso, a resposta é: no método `remove` da `HabitosStore` — o botão só chama ele.

Perguntas prováveis, com resposta pronta

Pergunta	Resposta curta
Por que dividir em <code>ui/</code> , <code>dominio/</code> , <code>dados/</code> ?	Pra decidir quem pode conhecer quem. Trocar a tela não afeta o domínio; trocar a origem do dado não afeta a tela.
Onde o dado realmente fica guardado?	Em <code>_memoria</code> , uma lista privada dentro de <code>HabitosRepositorio</code> (<code>dados/</code>) — hoje é só RAM.
Por que a <code>HabitosStore</code> não guarda a lista direto?	Ela delega a "fonte da verdade" ao repositório: depois de qualquer mudança, ela sempre relê tudo de lá (<code>await _repo.carregar()</code>).
Por que <code>watch</code> na lista e <code>read</code> no botão?	<code>watch</code> (no <code>build</code>) inscreve a tela pra ser redesenhada quando a store avisar; <code>read</code> (em <code>onPressed</code> / <code>initState</code>) só pega a store pra mandar uma ação, sem se inscrever.
O que é <code>notifyListeners()</code> ?	O aviso que a <code>ChangeNotifier</code> manda pra quem está "escutando" ela — equivalente ao <code>setState</code> , só que fora de uma tela.
Por que <code>remove</code> recebe o objeto <code>Habito</code> , não um índice?	Porque a tela de detalhe só tem o objeto — não sabe a posição dele na lista de origem.
Onde a exclusão acontece de verdade?	No método <code>remove</code> da <code>HabitosStore</code> . O <code>IconButton</code> só chama ele.
Por que <code>async</code> / <code>await</code> em tudo?	Simula acesso real a dados (banco, servidor) — a assinatura já fica pronta pra isso, mesmo sendo instantâneo hoje.
Para que serve <code>if (!context.mounted) return;</code> ?	Depois de um <code>await</code> , a tela pode ter sido fechada nesse intervalo. Essa linha evita usar um <code>context</code> que não existe mais.
O que muda se o app trocar de memória para banco de dados?	Só o corpo dos métodos de <code>HabitosRepositorio</code> . Nenhuma tela, nenhuma regra do domínio precisa mudar.